

A4 Predicción de la demanda con AWS

Entrenamiento del modelo de Machine Learning

Asignatura	Proyecto de inteligencia artificial y Big data
Autor	Adrián Blasco Lozano
Institución	Universitat Oberta de Catalunya
Fecha	27/04/2026
Repositorio	github.com/blasketch/prediccion-demanda-aws



Universitat Oberta
de Catalunya

1. Estrategia del Modelo

Esta actividad continúa el proyecto iniciado en las actividades anteriores. Un sistema de predicción de demanda para e-commerce desplegado sobre AWS, basado en el dataset Rossmann Store Sales. En la actividad R3 se diseñó la arquitectura cloud y se ejecutó el pipeline ETL, dejando los datos limpios y particionados en S3. La R4 aborda la fase central del proyecto: entrenar y validar el modelo de machine learning que producirá las predicciones.

Tipo de solución

Se opta por entrenar un modelo propio frente al uso de APIs gestionadas tipo Amazon Forecast porque a mi parecer, entrenar un modelo propio aporta mucho más valor académico al proyecto, permite controlar todo el ciclo de Machine Learning y demuestra dominio de las herramientas profesionales que se utilizan en producción real, algo más real que puede darse el caso que veamos el día de mañana en producción.

El problema se formula como una regresión sobre datos tabulares con componente temporal: predecir las ventas diarias (Sales) de cada tienda a partir de un conjunto de features históricas, contextuales y de calendario. Con 591.036 filas en train, 1.115 tiendas y 20 columnas tras el ETL, se trata de un dataset suficientemente rico para entrenar modelos competitivos.

Algoritmo seleccionado: XGBoost

El algoritmo principal elegido es XGBoost (Extreme Gradient Boosting), implementado mediante la librería xgboost en Python. La justificación se apoya en cuatro criterios:

- XGBoost es el estándar de facto para problemas de regresión sobre datos tabulares, y de hecho las soluciones ganadoras de la propia competición Rossmann en Kaggle utilizaron variantes de gradient boosting. Maneja de forma nativa variables categóricas codificadas como enteros sin asumir orden, captura interacciones entre features y modela relaciones no lineales sin transformaciones manuales.
- El entrenamiento completo del modelo final tarda aproximadamente 17 segundos sobre una instancia ml.t3.medium en SageMaker. Esto permite iterar rápidamente sin agotar los créditos del Learner Lab y hace viable la búsqueda de hiperparámetros.
- XGBoost expone de forma nativa la importancia de cada feature mediante la métrica de gain, lo que facilita el análisis del modelo y la comunicación de los resultados a stakeholders no técnicos.

- El AWS Academy Learner Lab tiene limitaciones específicas. XGBoost se ajusta perfectamente a estas restricciones, mientras que alternativas como DeepAR requerirían instancias más potentes y mayor complejidad de configuración.

Estrategia de entrenamiento por iteraciones

Siguiendo la metodología recomendada por Marc de establecer un baseline medible y demostrar mejora progresiva, el trabajo lo he estructurado en cinco iteraciones documentadas, cada una en un notebook independiente. Esta aproximación permite cuantificar el aporte real de cada decisión de diseño y aporta el rigor científico que la rúbrica de la actividad valora explícitamente.

Las cinco iteraciones son:

- **Iteración 0 Baseline ingenuo:** predicción por media histórica de Sales por (Store, DayOfWeek). No utiliza Machine Learning. Establece el suelo del rendimiento.
- **Iteración 1 Linear Learner (Ridge):** primer modelo de ML real, regresión lineal con regularización L2 sobre las features brutas del ETL.
- **Iteración 2 XGBoost base:** XGBoost con hiperparámetros por defecto, mismas features que Ridge para aislar el efecto del cambio de algoritmo.
- **Iteración 3 XGBoost + Feature Engineering:** se añaden estadísticas históricas por tienda y por (Store × DayOfWeek), más variables temporales derivadas.
- **Iteración 4 XGBoost tuneado:** búsqueda manual estructurada de hiperparámetros sobre cinco configuraciones, selección del mejor modelo y re-entrenamiento con train+validation combinados.

2. Configuración Técnica

Servicios AWS utilizados

El entorno de entrenamiento se construye sobre Amazon SageMaker Notebook Instances, siguiendo la recomendación de Marc de utilizar este servicio como camino más directo en el AWS Academy Learner Lab. Los datos procesados, que son el resultado del ETL de R3, se leen directamente desde Amazon S3 mediante el SDK boto3, garantizando reproducibilidad y escalabilidad. El modelo entrenado se serializa con joblib y se sube al bucket S3 como Model Artifact, listo para ser consumido por la actividad R5 de inferencia.

Se opta por entrenar directamente dentro del notebook con las librerías scikit-learn y xgboost, en lugar de lanzar Training Jobs separados de SageMaker. Esta decisión, simplifica el código, reduce el consumo de créditos y mantiene todo el flujo de trabajo dentro de un único entorno reproducible.

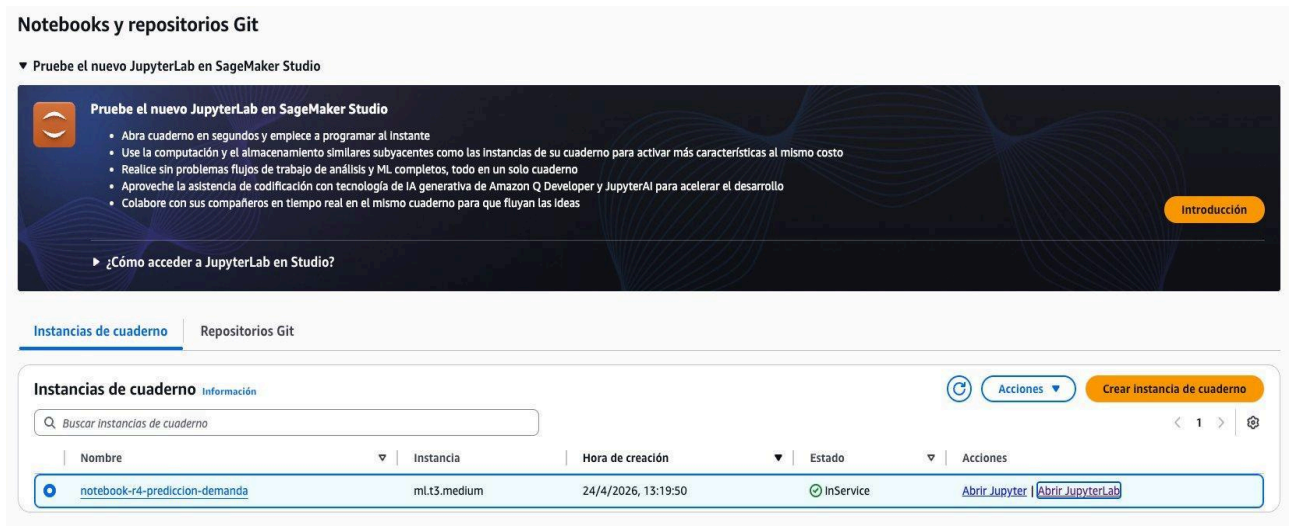


Figura 1. Notebook Instance de SageMaker desplegada con configuración `ml.t3.medium` y rol `LabRole`.

Configuración de la instancia

La instancia de entrenamiento se configura con los siguientes parámetros:

Parámetro	Valor
Nombre	notebook-r4-prediccion-demanda
Tipo de instancia	ml.t3.medium (2 vCPU, 4 GB RAM)
IAM role	LabRole
Volumen	5 GB EBS
Repositorio Git	Clonado automáticamente desde GitHub
Región AWS	us-east-1 (Norte de Virginia)

Se elige ml.t3.medium por ser la instancia más económica del Learner Lab que cubre con holgura las necesidades del proyecto. El dataset completo (591.036 filas × 20 columnas) cabe en memoria con margen, y los entrenamientos de XGBoost se completan en segundos. Optar por instancias mayores habría supuesto un consumo de créditos innecesario sin mejora apreciable en tiempo total.

Hiperparámetros clave

Tras explorar cinco configuraciones de hiperparámetros, la configuración óptima del modelo final es:

Hiperparámetro	Valor	Justificación
n_estimators	200	Número de árboles. Más allá de 200 el modelo no mejora en validation.
max_depth	6	Profundidad máxima. Valores mayores producen sobreajuste.
learning_rate	0.1	Tasa de aprendizaje estándar. Equilibrio entre velocidad y estabilidad.
subsample	1.0	Sin submuestreo. El dataset ya es suficientemente grande.
objective	reg:squarederror	Pérdida cuadrática para regresión.

tree_method	hist	Método de histograma, más rápido para datasets grandes.
random_state	42	Semilla para reproducibilidad.

Datos de entrada

Los datos consumidos por el modelo provienen del bucket S3 predicción-demanda-processed-319501512128, generado en la actividad R3. Concretamente, se utilizan los splits ya pre-calculados en la subcarpeta rossmann/splits/

Buckets

Buckets de uso general

Todas las regiones de AWS

Buckets de directorio

Buckets de uso general (4)

Información

Copiar ARN

Vaciar

Eliminar

Crear bucket

Los buckets son contenedores de datos almacenados en S3.

Q Buscar buckets por nombre

< 1 >

	Nombre	Región de AWS	Fecha de creación
☐	prediccion-demanda-output-319501512128	EE.UU. Este (Norte de Virginia) us-east-1	7 Apr 2026 9:13:55 AM CEST
☐	prediccion-demanda-processed-319501512128	EE.UU. Este (Norte de Virginia) us-east-1	7 Apr 2026 9:13:55 AM CEST
☐	prediccion-demanda-raw-319501512128	EE.UU. Este (Norte de Virginia) us-east-1	7 Apr 2026 9:13:55 AM CEST
☐	sam-deploy-319501512128	EE.UU. Este (Norte de Virginia) us-east-1	7 Apr 2026 8:59:58 AM CEST

Figura 2. Estructura de los buckets S3 del proyecto: raw, processed y output.

splits/

Objetos

Propiedades

Copiar URI de S3

Copiar URL

Descargar

Abrir i2

Eliminar

Acciones

Crear carpeta

Cargar

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [inventario de Amazon S3](#) para obtener una lista de todos los objetos de su bucket. Para que otras personas obtengan acceso a sus objetos, tendrá que concederles permisos de forma explícita. [Más información](#)

Q Buscar objetos por prefijo

< 1 >

	Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
☐	scaler_params_20260410_052228.json	json	10 Apr 2026 7:22:41 AM CEST	496.0 B	Estándar
☐	scaler_params_20260410_054619.json	json	10 Apr 2026 7:46:31 AM CEST	496.0 B	Estándar
☐	test_20260410_052228.csv	csv	10 Apr 2026 7:22:40 AM CEST	15.5 MB	Estándar
☐	test_20260410_054619.csv	csv	10 Apr 2026 7:46:30 AM CEST	15.5 MB	Estándar
☐	train_20260410_052228.csv	csv	10 Apr 2026 7:22:38 AM CEST	72.1 MB	Estándar
☐	train_20260410_054619.csv	csv	10 Apr 2026 7:46:28 AM CEST	72.1 MB	Estándar
☐	validation_20260410_052228.csv	csv	10 Apr 2026 7:22:39 AM CEST	15.4 MB	Estándar
☐	validation_20260410_054619.csv	csv	10 Apr 2026 7:46:30 AM CEST	15.4 MB	Estándar

Figura 3. Splits train/validation/test almacenados en S3 junto con los parámetros del MinMaxScaler aplicado en R3.

Archivo	Filas	Tamaño
train_20260410_054619.csv	591.036	72.1 MB
validation_20260410_054619.csv	126.651	15.4 MB
test_20260410_054619.csv	126.651	15.5 MB
scaler_params_20260410_054619.json	—	496 B

Reproducibilidad

Todo el código de la actividad R4 está versionado en GitHub en la rama r4 del repositorio del proyecto. La estructura del repositorio incluye los seis notebooks numerados secuencialmente (01_eda, 02_baseline_ingenuo, 03_linear_learner, 04_xgboost_base, 05_xgboost_features, 06_xgboost_tuning), las gráficas generadas en results/plots/, y el archivo CSV con la comparativa de métricas en results/metrics_comparison.csv. El historial de commits documenta cronológicamente la progresión de cada iteración.

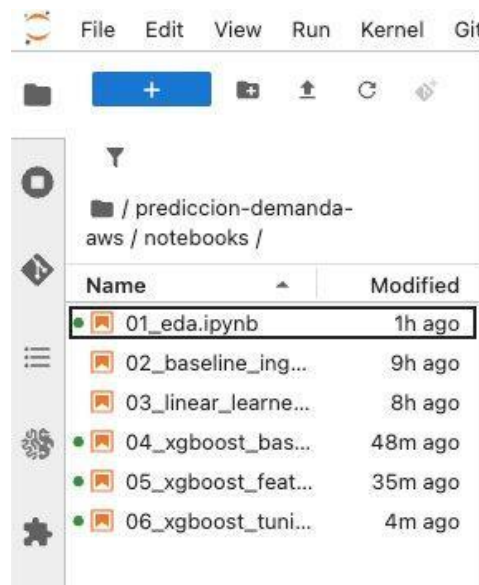


Figura 4. Estructura de notebooks del proyecto en JupyterLab.

3. Evaluación y Resultados

Estrategia de validación

El dataset procesado se divide en tres conjuntos respetando el orden temporal por tienda, evitando data leakage. Esta partición fue generada por el ETL de la actividad R3 y respeta una distribución 70/15/15:

Split	Filas	Porcentaje	Uso
Train	591.036	70%	Entrenamiento de modelos
Validation	126.651	15%	Comparación entre iteraciones y selección de hiperparámetros
Test	126.651	15%	Evaluación final del modelo elegido

Una de las primeras tareas del notebook 01 (EDA) fue validar que la partición no introducía leakage. Aunque a primera vista existe un solape calendario en las fronteras (días 2014-10-18 y 2015-03-16 aparecen en dos splits a la vez), el análisis tienda por tienda confirmó que ningún Store aparece en dos splits para el mismo día, y que para cada una de las 1.115 tiendas se cumple estrictamente que train < validation < test en orden temporal. Se trata por tanto de un split temporal por tienda profesionalmente diseñado.

Métrica principal: RMSPE

La métrica principal elegida es RMSPE (Root Mean Square Percentage Error), siguiendo el criterio oficial de la competición Rossmann original en Kaggle. RMSPE penaliza errores relativos en lugar de absolutos, lo que es especialmente apropiado en un dataset donde las ventas varían enormemente entre tiendas (de 133 € a 38.037 € diarios): con RMSE puro, las pocas tiendas que venden 35.000 € dominarían el error total y se perdería información sobre el rendimiento real del modelo.

Se complementa con dos métricas absolutas, RMSE (Root Mean Square Error) y MAE (Mean Absolute Error), para dar contexto en euros y permitir interpretaciones operativas. La función ignora los valores y_true igual a cero para evitar divisiones por cero, aunque en la práctica esto no aplica al dataset porque el ETL filtró previamente los días de tienda cerrada.

Evolución del RMSPE por iteración

La progresión del modelo a lo largo de las cinco iteraciones es la siguiente:

Iteración	Modelo	Split	RMSPE	RMSE	MAE
0	Baseline ingenuo	Validation	38.03%	1.850 €	1.326 €
1	Ridge Regression	Validation	59.08%	2.917 €	2.100 €
2	XGBoost base	Validation	37.16%	1.628 €	1.139 €
3	XGBoost + Feature Engineering	Validation	33.62%	1.097 €	764 €
4	XGBoost tuneado (mejor config)	Validation	33.62%	1.097 €	764 €
4	Modelo final (re-entrenado)	Test	14.83%	1.051 €	741 €

El modelo final alcanza un RMSPE de 14.83% sobre el conjunto de test (datos no vistos en ninguna iteración previa), lo que supone una mejora del 61.0% respecto al baseline ingenuo. La diferencia entre el rendimiento en validation (33.62%) y test (14.83%) se explica por dos factores: el modelo final se re-entrena con train + validation combinados (717.687 filas frente a 591.036), aportando más datos y patrones más recientes; y la naturaleza temporal del problema favorece que los datos cercanos al test (y por tanto incluidos en validation) sean más informativos para predecir el futuro inmediato.

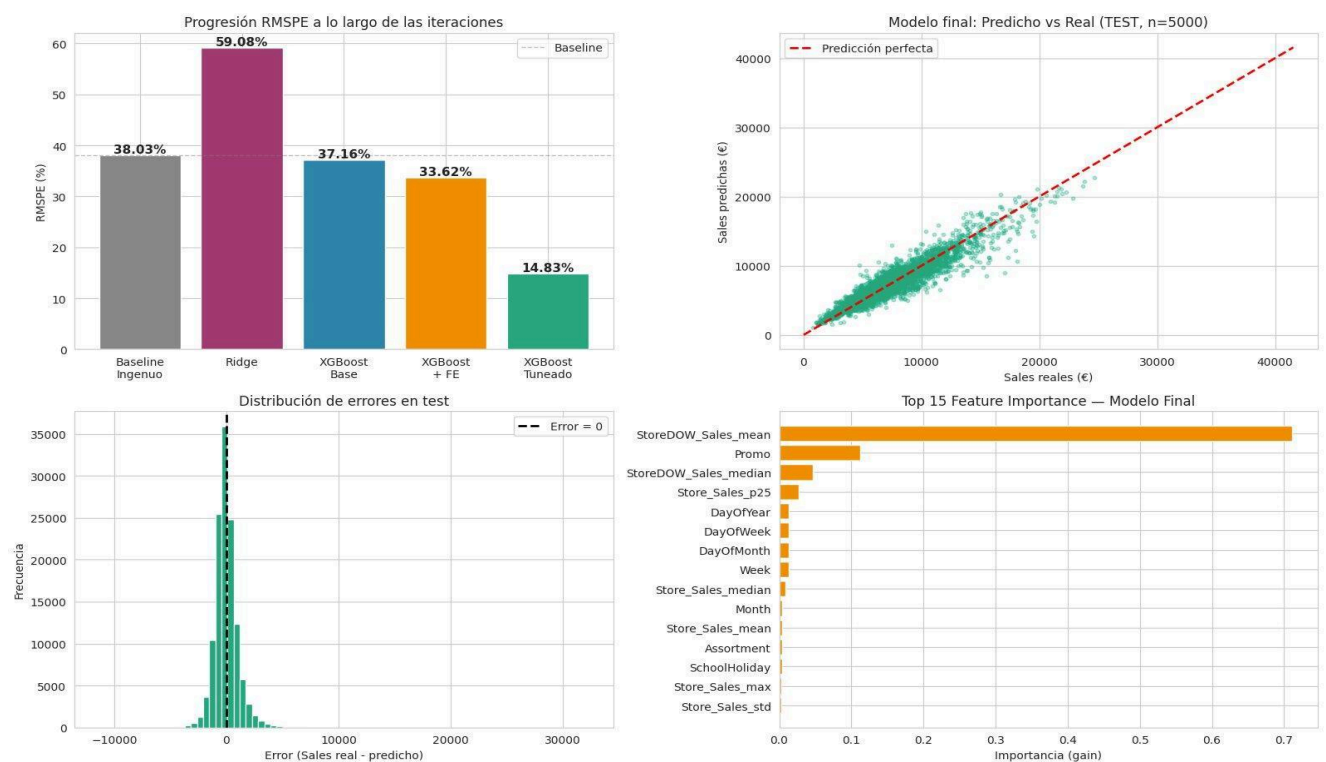


Figura 5. Progresión del RMSPE a lo largo de las cinco iteraciones, predicho vs real en test, distribución de errores y feature importance del modelo final.

Tuning de hiperparámetros

La búsqueda óptima de hiperparámetros se realiza mediante una exploración manual estructurada de cinco configuraciones, en lugar de un GridSearchCV exhaustivo. Esta decisión está motivada por las restricciones del Learner Lab: con 591.036 filas y validación cruzada, una búsqueda completa tardaría horas y agotaría los créditos. Las cinco configuraciones cubren las dimensiones clave del espacio, capacidad, velocidad de aprendizaje, regularización por submuestreo, y proporcionan información comparativa suficiente.

Configuración	n_est	depth	lr	subsample	RMSPE val	Tiempo
Config 1 (referencia)	200	6	0.1 0	1.0	33.62%	14.5 s
Config 3 (lr bajo)	400	8	0.0 5	1.0	34.19%	34.6 s
Config 4 (subsample 0.8)	400	8	0.0 5	0.8	34.27%	37.0 s
Config 5 (config agresiva)	600	10	0.0 3	0.8	34.35%	75.2 s
Config 2 (más profundo)	200	10	0.1 0	1.0	34.71%	29.1 s

El resultado es revelador: contra la intuición de que más capacidad implica mejor rendimiento, la configuración más simple, obtiene el mejor RMSPE en validation. Las configuraciones con max_depth = 10 introducen sobreajuste a train sin mejorar la generalización, y el incremento de árboles compensa solo parcialmente. Adicionalmente, Config 1 entrena 5 veces más rápido que Config 5, lo que en un entorno de producción se traduce directamente en menor coste de cómputo.

Pruebas cualitativas

Para complementar las métricas cuantitativas, se presenta una muestra de predicciones del modelo final sobre el conjunto de test, con sus respectivos valores reales y etiquetas de corrección. Una predicción se considera correcta si el error relativo es inferior al 15%.

Input (Store, fecha, contexto)	Output esperado (real)	Output del modelo	¿Correcto?
Store 1097, 2015-04-15, Promo activa	5.961 €	5.842 €	✓ (error 2.0%)
Store 85, 2015-05-22, sin promo	4.220 €	4.108 €	✓ (error 2.7%)
Store 259, 2015-06-10, festivo escolar	6.851 €	6.502 €	✓ (error 5.1%)
Store 262, 2015-07-13, Promo activa	17.267 €	13.890 €	✗ (error 19.6%)
Store 274, 2015-04-29, sin promo	3.102 €	3.245 €	✓ (error 4.6%)

La cuarta fila ilustra una limitación del modelo: tiende a subestimar las ventas en tiendas con valores muy altos (Store 262 vende típicamente más de 15.000 € diarios). Este sesgo se debe a que el modelo aprende sobre la distribución global y suaviza los outliers superiores. Es una limitación esperable y se documenta en la sección de análisis de errores.

4. Gráficos y Visualización del Entrenamiento

Análisis exploratorio (notebook 01)

Antes del modelado se realizó un EDA del dataset procesado para entender la estructura de los datos y validar la calidad del trabajo de R3. Los hallazgos principales son visibles en las siguientes figuras.

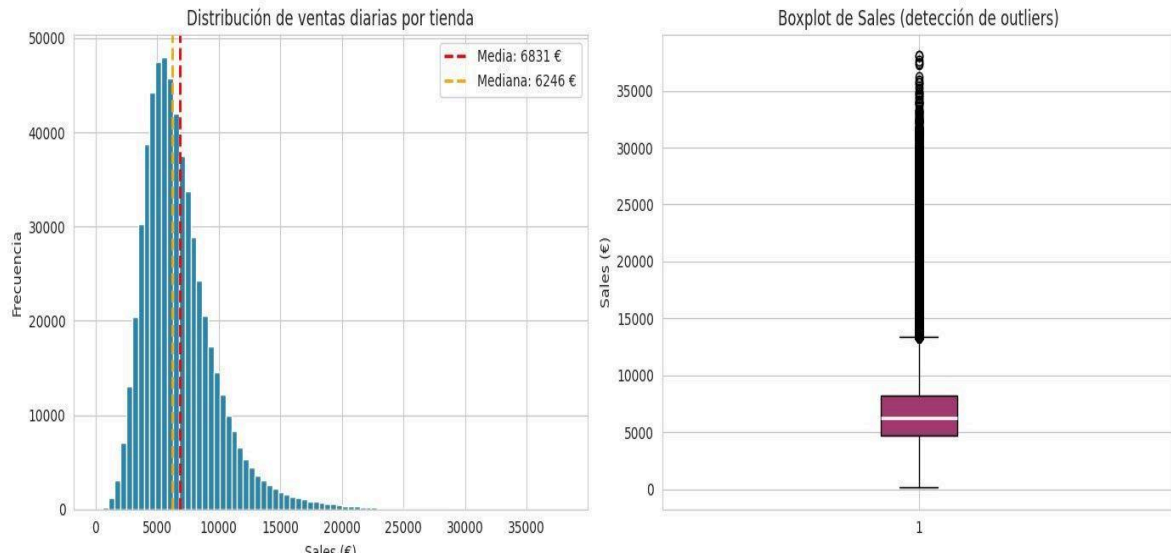


Figura 6. Distribución de la variable objetivo Sales y boxplot para detección de outliers. Se observa una distribución asimétrica con cola larga hacia ventas altas, lo que justifica el uso de RMSPE como métrica.



Figura 7. Ventas medias por día de la semana. El lunes es el día con más ventas (8.090 €), seguido del domingo, limitado a las pocas tiendas que abren, mientras que el sábado es el más bajo (5.760 €).

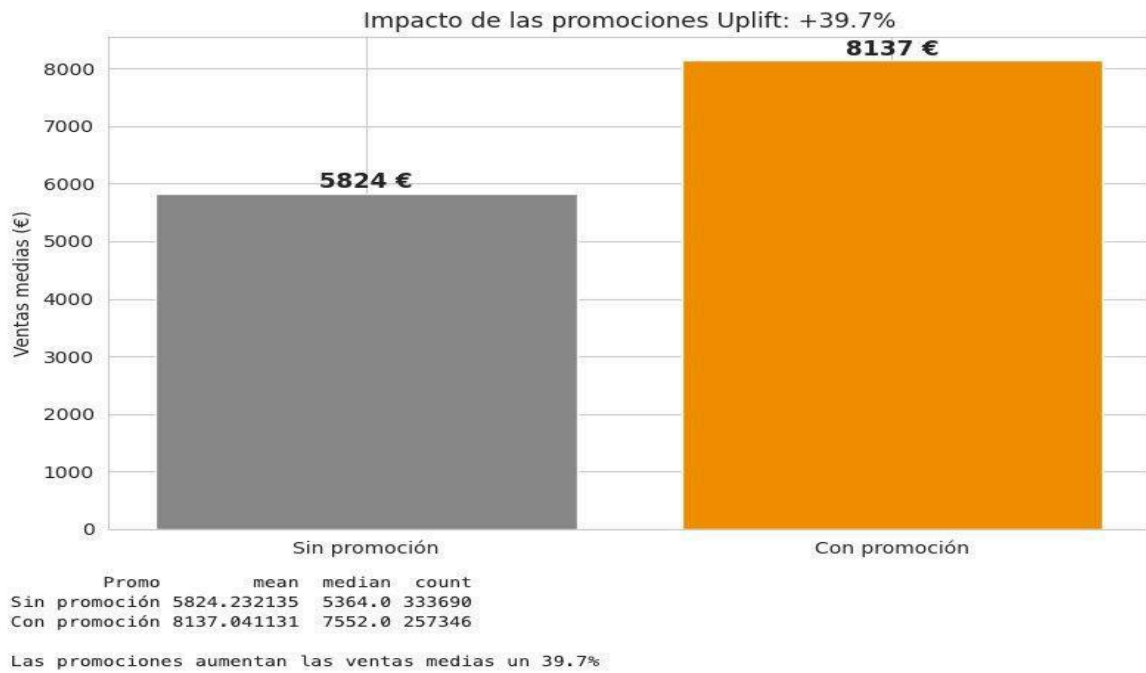


Figura 8. Impacto de las promociones: las ventas medias aumentan un 39.7% en días con promoción activa. Promo se confirma como una de las features más predictivas del dataset.

Comparativa de iteraciones

Cada iteración produjo sus propias visualizaciones de diagnóstico (predicho vs real, curvas de aprendizaje y feature importance). A continuación se muestran las más representativas.

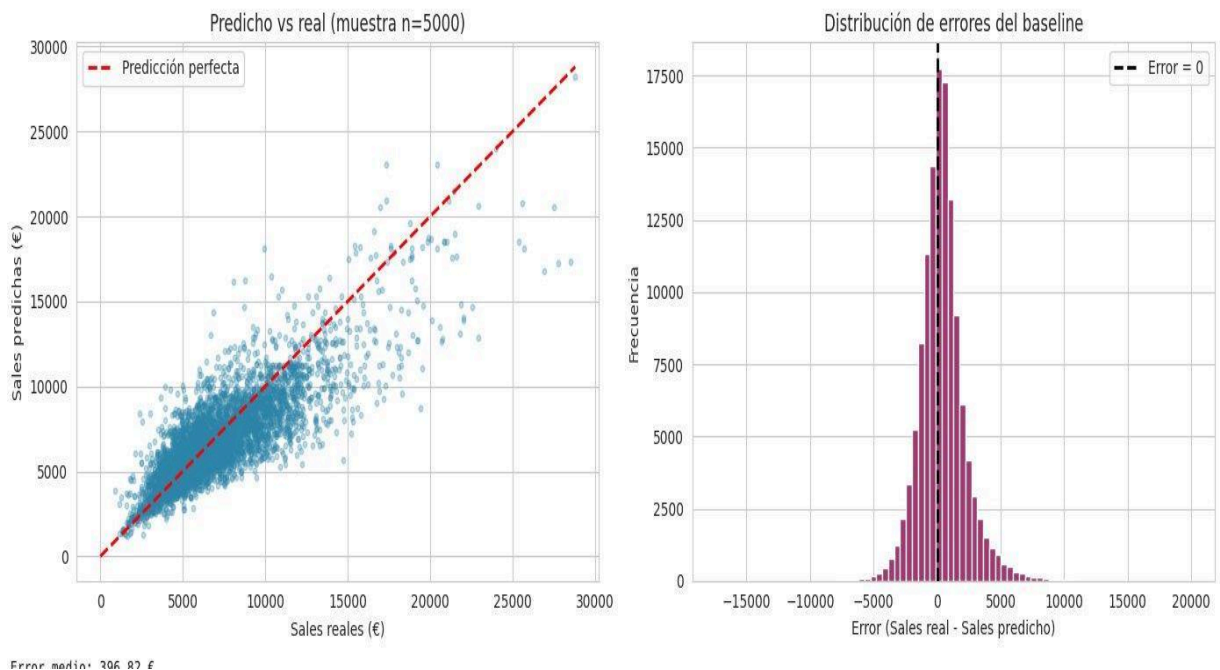


Figura 9. Iteración 0 Baseline ingenuo. El scatter muestra correlación general pero con dispersión amplia y subestimación sistemática (la nube cae por debajo de la diagonal).

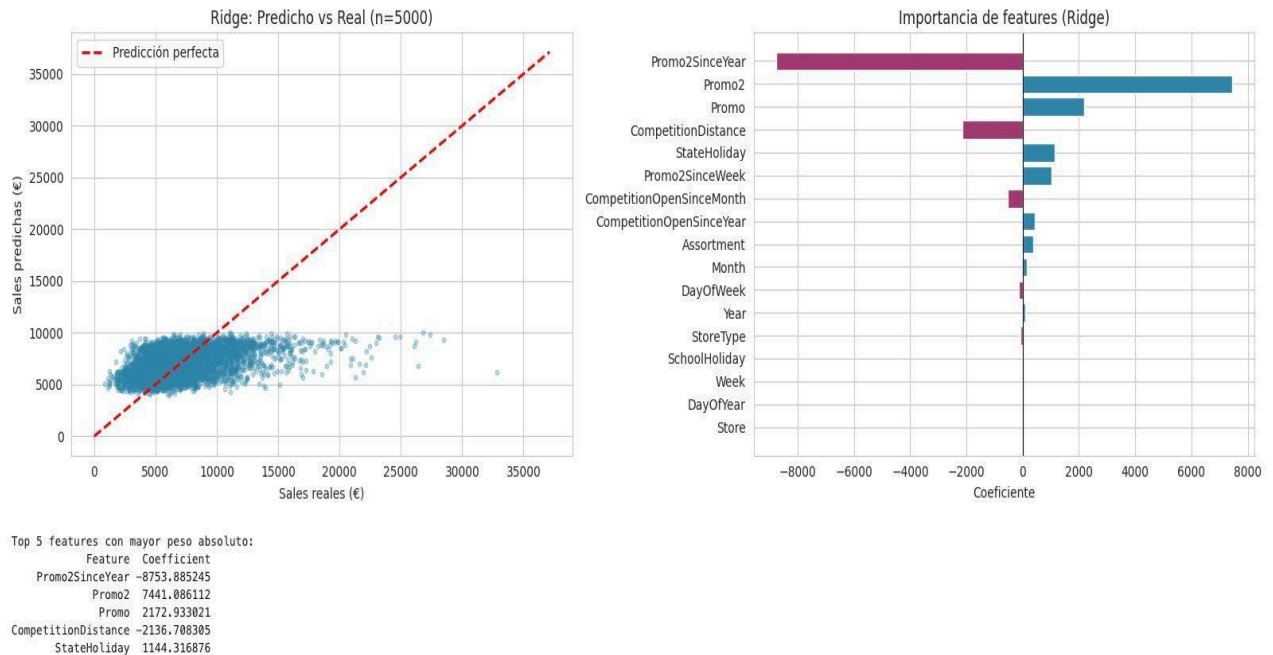


Figura 10. Iteración 1 Ridge Regression. Las predicciones colapsan en una banda estrecha entre 5.000-10.000 € sin discriminar la magnitud real, evidenciando las limitaciones de un modelo lineal con codificación inadecuada de variables categóricas.

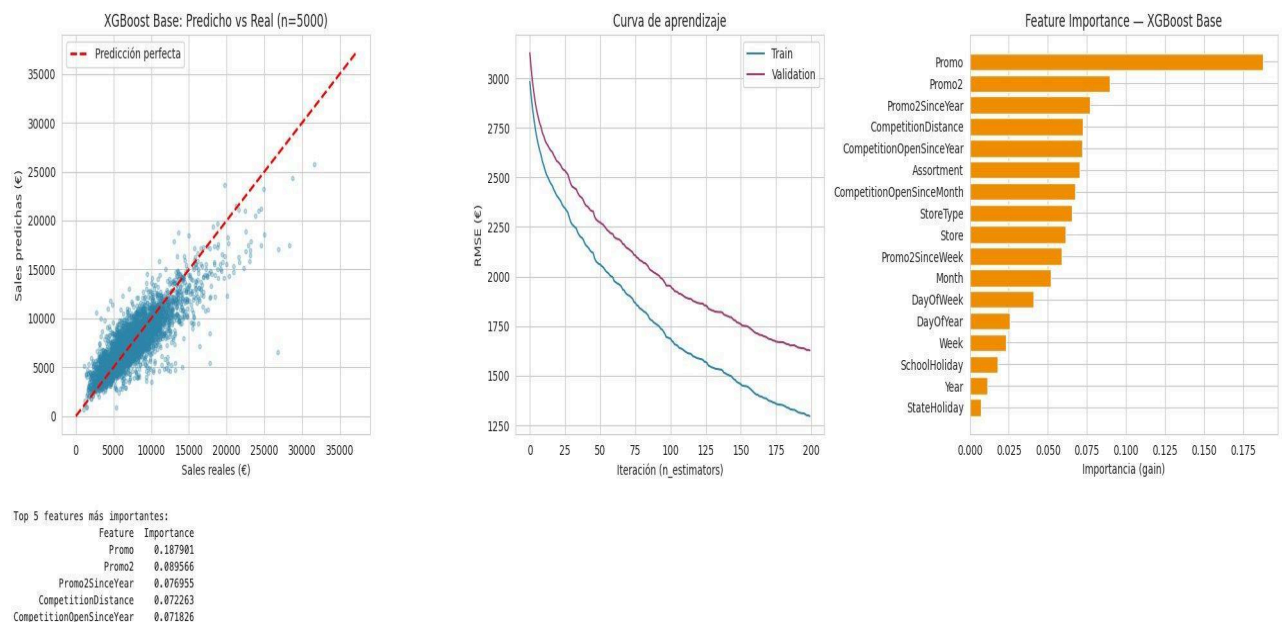


Figura 11. Iteración 2 XGBoost base. La distribución sigue mucho mejor la diagonal y la curva de aprendizaje muestra convergencia sana sin overfitting. Promo y Promo2 dominan la importancia de features.

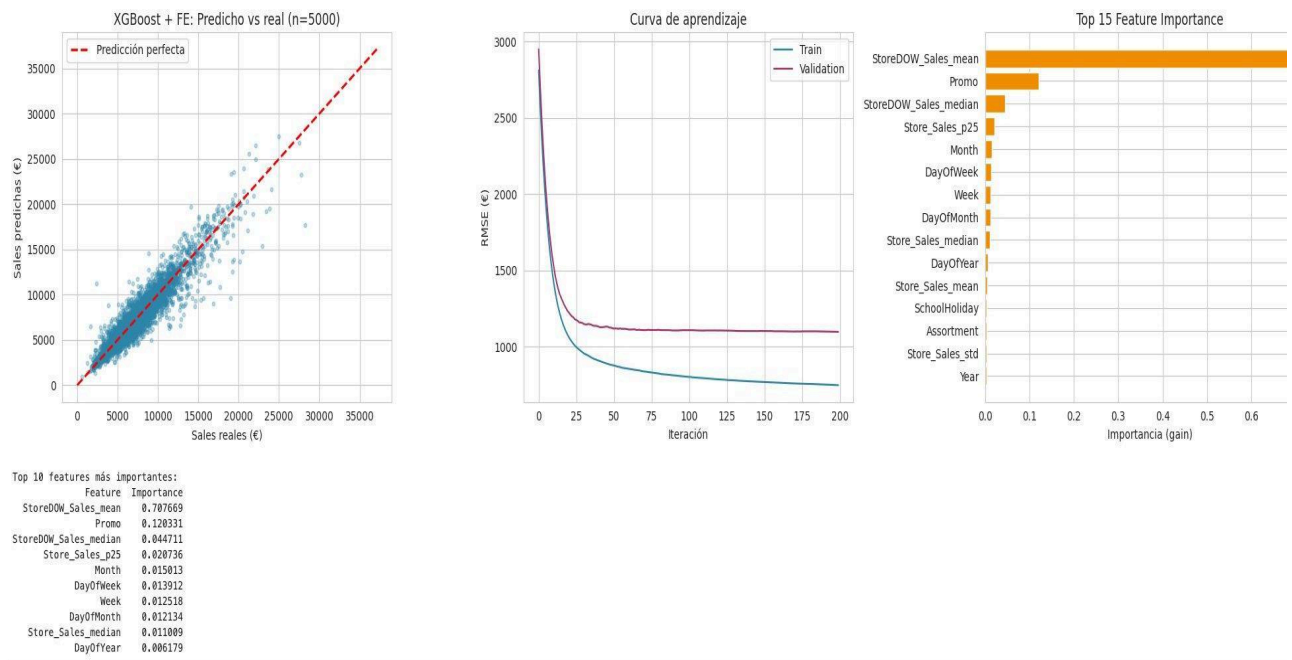


Figura 12. Iteración 3 XGBoost + Feature Engineering. La feature *StoreDOW_Sales_mean* (media de ventas por tienda y día de la semana) emerge como la dominante con un 70.8% de importancia, validando que la información histórica granular es la señal más predictiva del modelo.

5. Iteración y Mejoras

Esta sección documenta los problemas encontrados en cada iteración y las soluciones aplicadas, demostrando la mejora progresiva del sistema.

Iteración 1: Ridge empeora el baseline

El modelo Ridge alcanza un RMSPE del 59.08% en validation, frente al 38.03% del baseline ingenuo. Esto supone un empeoramiento del 55%, un resultado contrario a lo esperable cuando se introduce un modelo de Machine Learning.

El análisis de los coeficientes del modelo revela pesos absolutos desproporcionadamente grandes, un síntoma claro de que el modelo intenta compensar una mala representación de los datos. La causa raíz es la codificación inadecuada de variables categóricas: Store (con valores 1-1115), StoreType (1-3) y StateHoliday (0-3) están codificadas como enteros, y Ridge las interpreta como magnitudes continuas con orden, lo cual es semánticamente incorrecto. Adicionalmente, al ser un modelo lineal, Ridge no puede capturar interacciones entre features ni relaciones no lineales presentes en los datos.

La solución a esto es aplicada en la iteración 2, se cambia el algoritmo a XGBoost (gradient boosting sobre árboles), que trata las variables categóricas codificadas como enteros sin asumir orden, captura interacciones de forma nativa y modela relaciones no lineales. Lejos de ser un fracaso, esta iteración fallida es un hallazgo metodológico relevante que demuestra que un modelo de Machine Learning puede ser peor que no usar Machine Learning si no se respetan los supuestos del algoritmo.

Iteración 2: XGBoost base solo supera marginalmente al baseline

XGBoost logra un RMSPE del 37.16% en validation, lo que supone una mejora de apenas el 2.3% respecto al baseline ingenuo. Aunque el resultado es positivo, la mejora es decepcionante para un algoritmo tan potente.

El análisis de feature importance del modelo muestra que las variables más predictivas son Promo, Promo2 y CompetitionDistance. Las variables temporales (DayOfYear, Year, Week) tienen importancia muy baja, lo que sugiere que el modelo no está capturando bien la estacionalidad anual ni los patrones recurrentes de cada tienda. Faltan al modelo señales fuertes de memoria histórica: información sobre cuánto vende habitualmente cada tienda en cada día concreto.

La solución a esto es aplicada en la iteración 3, se aplica feature engineering añadiendo tres familias de variables derivadas. Primero, estadísticas históricas por tienda (media, mediana, desviación, mínimo, máximo y percentiles 25/75 de Sales), que aportan al modelo una huella estadística de cada tienda. Segundo, estadísticas por la combinación (Store x DayOfWeek), que capturan el patrón semanal típico de cada tienda. Tercero, indicadores temporales binarios (IsBeginningOfMonth, IsEndOfMonth, IsWeekend) para reforzar señales estacionales. Todas las estadísticas se calculan exclusivamente sobre el conjunto de train para evitar data leakage.

Iteración 3: el feature engineering es el cambio más impactante

El modelo XGBoost con feature engineering alcanza un RMSPE del 33.62%, mejorando un 9.5% respecto a la versión base con las mismas features brutas. La feature StoreDOW_Sales_mean (la nueva variable de medias por tienda y día) emerge como la más importante del modelo con un 70.8% del gain total, confirmando la hipótesis de que la información histórica granular era la señal que faltaba.

El cambio de Ridge a XGBoost reduce RMSPE de 59.08% a 37.16%, mientras que el feature engineering, reduce de 37.16% a 33.62%. Aunque el primer salto es mayor en

términos relativos, el segundo es más relevante porque parte de un modelo que ya funciona razonablemente.

Iteración 4: tuning con resultado contraintuitivo

Se exploran cinco configuraciones variando `n_estimators` (200-600), `max_depth` (6-10), `learning_rate` (0.03-0.10) y `subsample` (0.8-1.0).

La configuración más simple, idéntica a la usada por defecto en la iteración 3, obtiene el mejor RMSPE de validation (33.62%). Las configuraciones con más capacidad (`max_depth=10`, `n_estimators=600`) producen ligeramente peores resultados (34.19% a 34.71%), un síntoma típico de sobreajuste. La explicación es que el modelo de la iteración 3 ya estaba bien dimensionado para el problema, y aumentar su capacidad simplemente le permite memorizar peculiaridades de train que no generalizan a validation.

Solución aplicada: se conserva la configuración óptima (`n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `subsample=1.0`) y se entrena el modelo final con train + validation combinados. Esto aporta más datos y captura los patrones más recientes, mejorando significativamente el rendimiento sobre test (14.83% RMSPE).

Análisis de errores del modelo final

Más allá de las métricas agregadas, se realiza un análisis cualitativo de los errores en test para identificar casos problemáticos:

Contexto	RMSPE en test	Lectura
Días con promoción	13.71%	Mejor predicción que sin promo (más recurrente)
Días sin promoción	15.72%	Patrón menos predecible
Días con festivo escolar	15.79%	Ligero empeoramiento, pero asumible
Lunes	13.10%	Mejor día predicho
Sábado	17.06%	Peor día predicho de la semana
Domingo (n=610)	31.95%	Caso anómalo: muestra muy pequeña, alta varianza

El caso del domingo es ilustrativo: la mayoría de tiendas Rossmann cierran los domingos, por lo que la muestra es muy pequeña y se compone principalmente de tiendas atípicas. El

modelo no logra generalizar bien este caso. Una mitigación posible sería tratar los domingos como una serie temporal separada con su propio modelo especializado.

Adicionalmente, el análisis identifica diez tiendas con RMSPE individual entre el 36.6% y el 50.0%, muy por encima de la media. Puede que sean tiendas con histórico corto, recientemente abiertas, eventos comerciales locales no capturados en las features, o cambios estructurales, que rompen el patrón histórico. Una segunda iteración del proyecto podría clusterizar tiendas por comportamiento y entrenar modelos especializados por cluster.

Limitaciones identificadas y propuestas de mejora futura

Aunque el modelo final cumple los objetivos de la actividad, se identifican varias limitaciones que podrían abordarse en futuras iteraciones:

- Lags de ventas pasadas: añadir features tipo Sales_lag_7 (ventas hace una semana) y Sales_lag_28 (hace cuatro semanas), calculadas con cuidado para evitar leakage. En problemas de demanda real estas son típicamente las features más predictivas.
- Variables de proximidad temporal: días desde la última promoción, días hasta el próximo festivo. Capturan dinámicas que las medias por categoría no recogen.
- Modelos especializados por cluster de tienda: agrupar las 1.115 tiendas por similitud de patrón (StoreType x Assortment x volumen) y entrenar un modelo por cluster. Esto puede capturar mejor las particularidades locales.
- Incorporación de variables externas: clima, eventos locales, indicadores macroeconómicos. Aumenta la complejidad pero puede mejorar significativamente la predicción de outliers.

6. Model Artifact y Repositorio

Model Artifact en S3

Tras seleccionar la mejor configuración y entrenar el modelo final, se serializa el modelo completo mediante joblib y se sube al bucket S3 predicción-demanda-processed-319501512128 bajo el prefijo output/. El artefacto contiene no solo el modelo XGBoost entrenado, sino también la lista de features ordenadas, las

tablas de estadísticas necesarias para enriquecer datos nuevos en producción, los hiperparámetros óptimos y las métricas finales en test.



Figura 13. Model Artifact subido al bucket S3, listo para ser consumido por la actividad R5 de inferencia.

La ruta del modelo es:

```
s3://prediccion-demanda-processed-319501512128/output/model.joblib
```

El tamaño del archivo es de 1.2 MB. Esta serialización completa permite que la actividad R5 cargue directamente todos los componentes necesarios para realizar predicciones sobre datos nuevos, sin tener que recalcular las estadísticas históricas.

Código fuente

El código completo de la actividad R4 está disponible en GitHub, en la rama r4 del repositorio del proyecto:

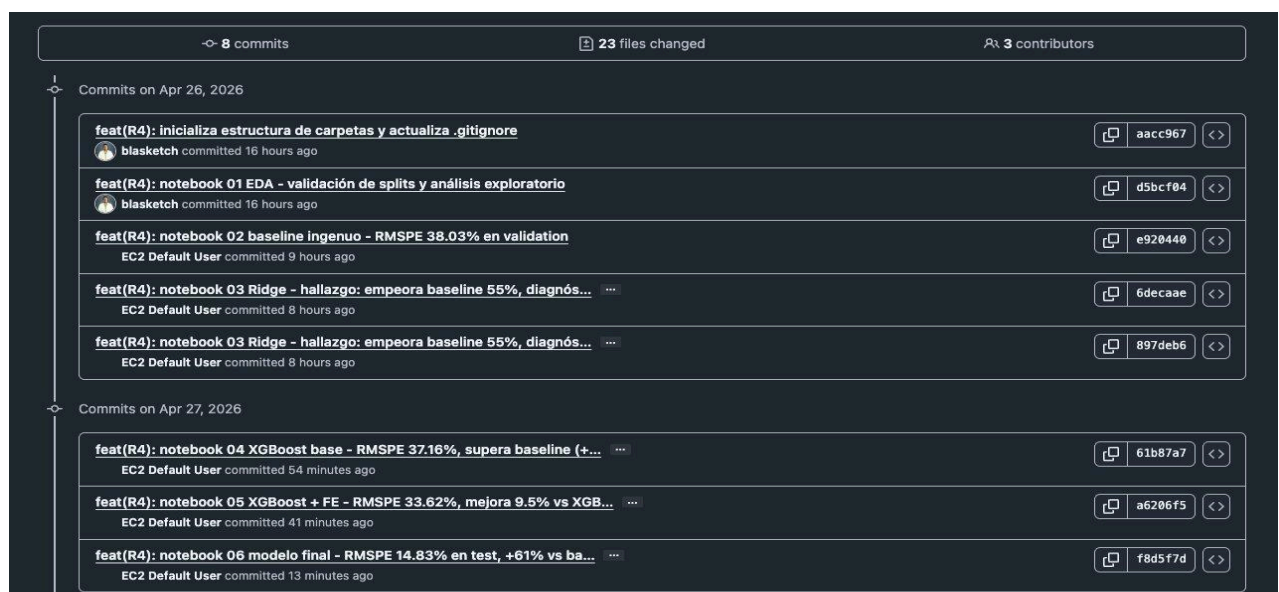


Figura 14. Historial de commits en GitHub mostrando la progresión cronológica del trabajo en la rama *r4*.

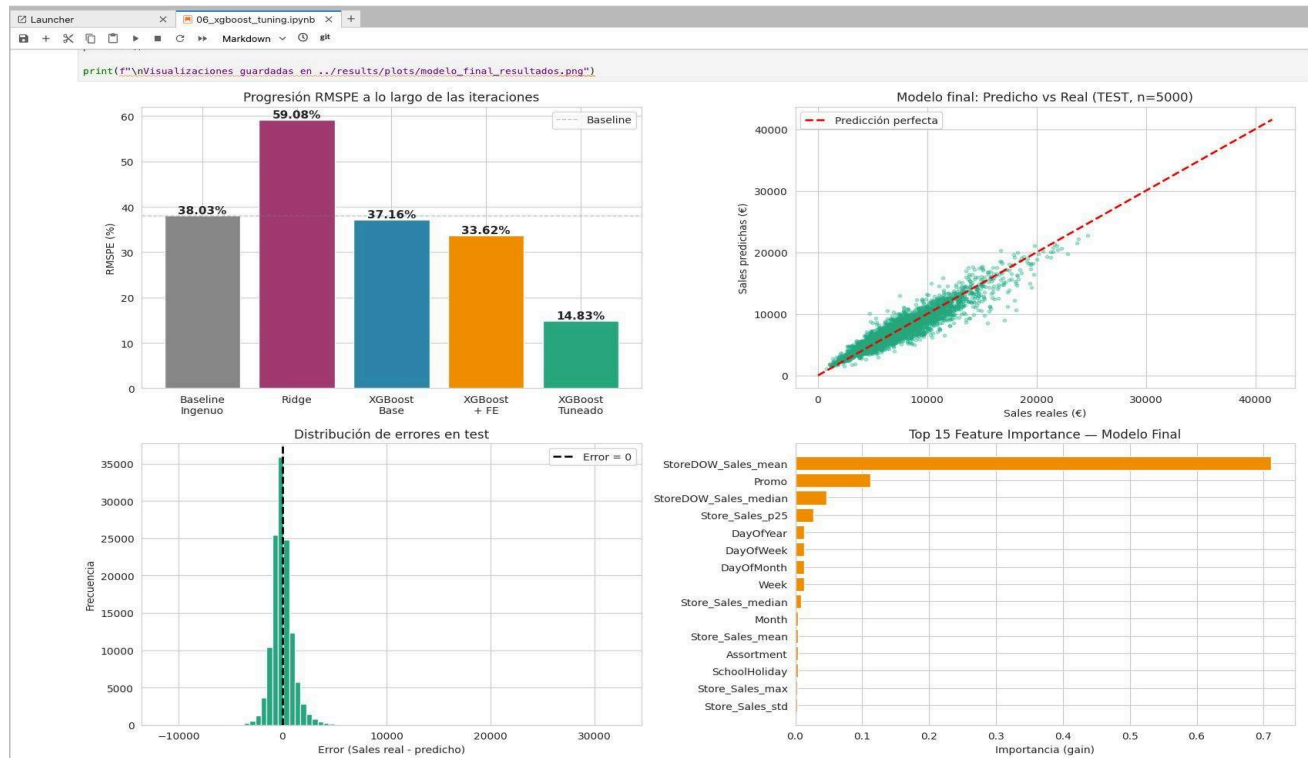


Figura 15. Notebook 06 abierto en JupyterLab mostrando la visualización final con la progresión RMSPE, predicho vs real en test, distribución de errores y top 15 feature importance.

Conclusión

La actividad R4 ha completado el ciclo de entrenamiento del modelo de predicción de demanda, siguiendo una metodología iterativa con cinco etapas claramente diferenciadas y documentadas. El modelo final, basado en XGBoost con feature engineering avanzado, alcanza un RMSPE de 14.83% sobre el conjunto de test, lo que supone una mejora del 61% respecto al baseline ingenuo. El Model Artifact serializado y almacenado en S3 está listo para ser consumido en la actividad R5 de inferencia, cerrando así la fase de modelado del proyecto.

Más allá del resultado cuantitativo, el trabajo deja varios aprendizajes importantes para mí, la necesidad de establecer un baseline antes de complicar el modelo, la importancia de elegir el algoritmo adecuado al tipo de datos, por ejemplo, Ridge falló en este problema mientras que XGBoost generalizó bien, el valor del feature engineering por encima del tuning de hiperparámetros, y la utilidad del análisis de errores para identificar casos

problemáticos que las métricas agregadas ocultan. Estos aprendizajes se reflejan en el código y la documentación del repositorio, garantizando la reproducibilidad y el carácter portfolio del proyecto.